MULTIMEDIA UNIVERSITY®

Inquire, Inspire and Innovate

FACULTY OF COMPUTING & INFORMATICS

(FCI)

TRIMESTER 2610

CNS6214 OPERATING SYSTEMS

ASSIGNMENT REPORT

TT1L

NAME	STUDENT ID	EMAIL
JASON TAN JIA SHENG	242UC244KW	jason.tan.jia@student.mmu.edu.my
CHONG JIA ZHEN	251UC250CU	chong.jia.zhen@student.mmu.edu.my
ATODAN RIZKY PARIJOGO	243UC246PD	atodan.rizky.parijogo@student.mmu.edu.my

Architecture Diagram

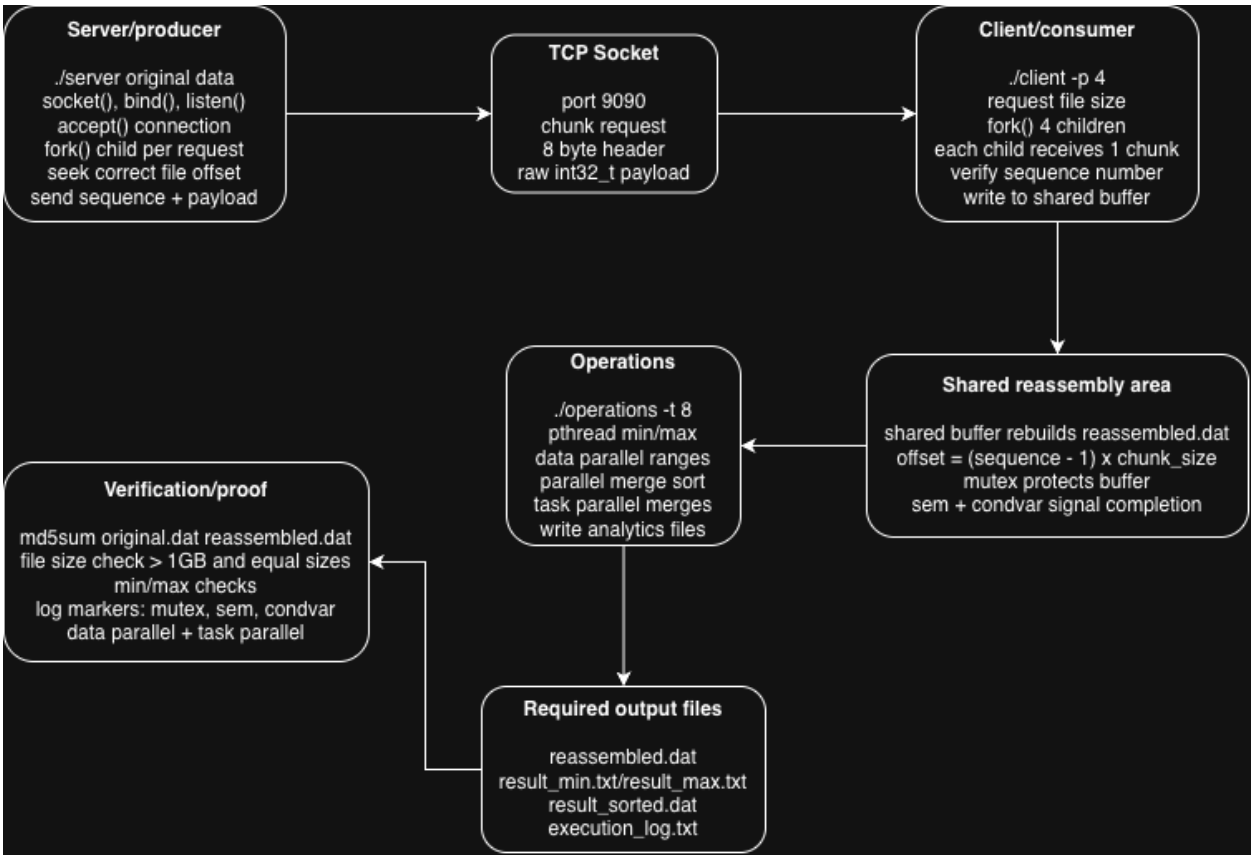


Figure 1: Architecture diagram

The system has 3 important parts such as server, client and operations. The server is the producer. It will be listening on TCP port 9090, accepting incoming connections, and forks a child process for each chunk request. Each child is receiving the requested chunk number and seeks to the correct offset in original.dat, then it will send an 8-byte header followed by raw int32_t payload data.

The client is the consumer. It will be running as ./client -p 4, requesting the file size, allocating a shared mmap reassembly buffer, then forks 4 children. Each child requests one chunk, verifies the sequence number and writes the payload into the correct location. After successful reassembly, the client will write the reassembled.dat and launches Part 2 using fork and exec.

The operations program will need to read reassembled.dat, compute the minimum and maximum with worker threads, perform the parallel merge sort and write the required output files.

Synchronization Strategy

1. Mutex (pthread_mutex_t)

Used:

Part 1: Protects the shared reassembly buffer counters (chunks_received, chunks_finished)

Part 2: Protects global_min and global_max during reduction phase

Why needed:

Without mutex protection, two threads could read the same old value of global_min, both compute a new minimum, and write back - causing one update to be lost (race condition).

Implementation (operations.c) :

```
/* NOTE: The mutex protects the shared global_min and finished_threads
       values; the condition variable wakes the main thread after reductions. */
    pthread_mutex_lock(&result_lock);
    if (local_min < global_min) {
        global_min = local_min;
    }
    finished_threads++;
    pthread_cond_signal(&done_cond);
    pthread_mutex_unlock(&result_lock);
```

2. Semaphore (sem_t)

Used:

Part 1 (Client): The received_count semaphore counts completed chunk transfers. Initialized to 0, each child calls sem_post() when its chunk arrives. The main process calls sem_wait() N times to ensure all chunks are received before writing the file.

Part 2 (Operations): The sort_slots (or merge_sem) semaphore limits how many merge tasks can execute their memory-intensive merge work concurrently. This acts as a **workload throttle** to prevent thread explosion.

Why needed:

Part 1: The semaphore provides a counting mechanism that guarantees the main process waits for exactly N chunks to arrive, regardless of the order in which children complete.

Part 2: Without throttling, the bottom-up merge phase could spawn hundreds of merge threads simultaneously, causing excessive context-switching overhead and memory bandwidth contention. The semaphore ensures only T threads (where T is the user-specified thread count) perform merge work at any given time.

Implementation (operations.c) :

```
sem_wait(&sort_slots); // Acquire a "slot" (blocks if T threads busy)

merge_ranges(task->array, task->temp, task->left, task->mid, task->right);
// Do the memory-intensive merge work

sem_post(&sort_slots); // Release the slot for another thread
```

3. Condition Variable (pthread_cond_t)

Used:

Part 1 (Client): The chunk_arrived condition variable allows the main process to sleep efficiently while waiting for children to signal chunk completion, instead of busy-waiting in a loop.

Part 2 (Operations): The done_cond (or level_cond) coordinates the parallel merge sort phases. Worker threads signal when they finish their local work, and the main thread waits until all threads at level N complete before starting level N+1.

Why needed: Condition variables allow threads/processes to sleep efficiently instead of burning CPU cycles in a polling loop. When a thread calls pthread_cond_wait(), it atomically releases the mutex and sleeps. Another thread calls pthread_cond_signal() to wake it up when the condition is true. This is much more efficient than checking a flag repeatedly.

Implementation (operations.c):

```
/* Main thread waits for all workers */
    pthread_mutex_lock(&result_lock);
    while (finished_threads < thread_count) {
        pthread_cond_wait(&done_cond, &result_lock);
    }
    pthread_mutex_unlock(&result_lock);
```

Parallelism Implementation

The operations program uses data parallelism for minimum and maximum computation, and a combination of data and task parallelism for the sort. The number of threads T is controlled by the –t flag.

Data Parallelism (Minimum and Maximum)

To find the minimum and maximum, the array is divided into T equal blocks, one per thread. The division is calculated as:

```
for (i = 0; i < thread_count; i++) {
    tasks[i].array = array;
    tasks[i].start = count * (size_t)i / (size_t)thread_count;
    tasks[i].end = count * (size_t)(i + 1) / (size_t)thread_count;
```

The formula in this code ensures that each thread receives a proportional share of the array based on its index. The first thread starts at zero, and the last thread ends at the final element. With no gaps or overlaps between blocks. Each thread is then launched with pthread_create and runs find_min_worker, scanning only its assigned block to find a local minimum:

```
/* NOTE/DATA PARALLELISM: each thread scans one separate block of the array. */
for (i = task->start; i < task->end; i++) {
    if (task->array[i] < local_min) {
        local_min = task->array[i];
    }
}
```

Since no two threads ever access the same memory during the scan, all the T threads run simultaneously with no interference – this is data parallelism. Once a thread finishes scanning, it locks the mutex to safely compare and update the global result:

```
    /* NOTE: The mutex protects the shared global_min and finished_threads
       values; the condition variable wakes the main thread after reductions. */
    pthread_mutex_lock(&result_lock);
    if (local_min < global_min) {
        global_min = local_min;
    }
    finished_threads++;
    pthread_cond_signal(&done_cond);
    pthread_mutex_unlock(&result_lock);

    return NULL;
}
```

The lock is held only for a few lines, not during the scan itself, so it does not slow down the parallel work. The main thread sleeps on the condition variable until all threads have reported in. The same pattern runs again for the maximum for using find_max_worker.

Data Parallelism + Task Parallelism — Parallel Merge Sort

Phase A — Data Parallelism

The same workload division formula applies to the sort. Each thread receives an equal, non-overlapping chunk of the array and independently sorts it using sort_block_worker:

```
/* NOTE/DATA PARALLELISM: independent blocks are sorted by separate threads. */
qsort(task->array + task->start, task->end - task->start,
      sizeof(int32_t), compare_int32);
```

Since each thread operates on a completely separate region, synchronization is needed, and all T threads sort simultaneously – this is data parallelism. After Phase A, the array contains T separately sorted chunks sitting next to each other.

Phase B — Task Parallelism

Phase B merges adjacent pairs of sorted chunks in rounds, doubling the merged run size each round until the entire array is one fully sorted sequence. Each merge of a pair is treated as an independent task and given its own thread. Before creating each merged thread, sem_wait is called to ensure no more than T merge threads are active at the same time:

```
        sem_wait(&sort_slots);
        merge_tasks[i].array = array;
        merge_tasks[i].temp = temp;
        merge_tasks[i].left = left;
        merge_tasks[i].mid = mid;
        merge_tasks[i].right = right;

        if (pthread_create(&merge_threads[i], NULL, merge_worker,
                           &merge_tasks[i]) != 0) {
            fprintf(stderr, "pthread_create merge failed\n");
            sem_post(&sort_slots);
            free(merge_threads);
            free(merge_tasks);
            free(threads);
            free(sort_tasks);
            free(temp);
            return -1;
        }
        i++;
    }
```

Each merge_worker performs its merge, then signals completion and releases its semaphore plot:

```
static void *merge_worker(void *arg) {
    MergeTask *task = arg;

    /* NOTE/TASK PARALLELISM: each merge is an independent task in the merge phase. */
    merge_ranges(task->array, task->temp, task->left, task->mid, task->right);
    int pthread_cond_signal(pthread_cond_t *cv)
    Generate Copilot summary
    pthread_cond_signal(&done_cond);
    pthread_mutex_unlock(&result_lock);

    sem_post(&sort_slots);
    return NULL;
}
```

Since each task merges and a different pair of runs, all the tasks within a round run concurrently – this is task parallelism. The main thread waits on the condition variable between rounds to guarantee one round is fully complete before the next begins, preventing threads from reading data that is still being written.

Individual Contribution Matrix

Student	Modules / Functions Owned	Testing / Validation Role	% Contribution	Commit / Trace Evidence
Jason Tan Jia Sheng	Server producer, chunk division, fork management, file I/O, TCP response header	Server chunk transfer validation, socket stress test, error handling, MD5 integrity check	33.33%	.git history, server.c, terminal validation output
Chong Jia Zhen	Client child process, shared mmap buffer, synchronization (mutex/sem/condvar), reassembly logic	Chunk reordering validation, synchronization testing, process management verification	33.33%	.git history, client.c, strace output showing fork/wait
Atodan Rizky Parijogo	Parallel min/max, parallel merge sort, output file writers, execution log, main entry and argument parsing	Verified sorted output correctness, min/max accuracy, execution log format, and thread count scaling across T=1, 2, 4, 8	33.33%	.git history, operations.c, terminal output

Testing & Validation

Part 1

```
theusefulhacker@debian:~/project_root$ make
gcc -std=c99 -Wall -Wextra -O2 -D_GNU_SOURCE -D_FILE_OFFSET_BITS=64 -o server src/server.c
gcc -std=c99 -Wall -Wextra -O2 -D_GNU_SOURCE -D_FILE_OFFSET_BITS=64 -o client src/client.c -pthread
gcc -std=c99 -Wall -Wextra -O2 -D_GNU_SOURCE -D_FILE_OFFSET_BITS=64 -o operations src/operations.c -pthread
theusefulhacker@debian:~/project_root$
```

Figure 2: Build success

```
theusefulhacker@debian:~/project_root$ ./server original.dat
Server listening on port 9090
Sending chunk 3 of 4
Sending chunk 1 of 4
Sending chunk 4 of 4
Sending chunk 2 of 4
```

Figure 3: Server running and chunk sending

```
theusefulhacker@debian:~/project_root$ md5sum original.dat reassembled.dat
068b43c9da1c3de1733ce4e7c36306ce  original.dat
068b43c9da1c3de1733ce4e7c36306ce  reassembled.dat
theusefulhacker@debian:~/project_root$
```

Figure 4: File integrity

```
theusefulhacker@debian:~/project_root$ diff <(hexdump -C original.dat) <(hexdump -C reassembled.dat)
Killed
theusefulhacker@debian:~/project_root$ cmp -s original.dat reassembled.dat && echo "PASS: files are identical" || echo "FAIL: files differ"
PASS: files are identical
theusefulhacker@debian:~/project_root$
```

Figure 5: Chunk reordering correctness

The hexdump | diff method was killed because it converts a 1.08GB binary file into several GB of text twice, then diff compares the text streams. This uses much more memory than the original file size. md5sum and cmp are better validation methods because they compare the binary files directly without expanding them.

```
theusefulhacker@debian:~/project_root$ ls -lh original.dat reassembled.dat
-rw-rw-r-- 1 theusefulhacker theusefulhacker 1.1G Jun  5 16:32 original.dat
-rw-rw-r-- 1 theusefulhacker theusefulhacker 1.1G Jul  4 17:17 reassembled.dat
```

Figure 6: File size > 1GB

```
theusefulhacker@debian:~/project_root$ cat execution_log.txt
[PART1] CHUNKS=4 | PROCS=4 | SYNC_USED=mutex,sem,condvar
[PART2] THREADS=8 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=36320 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
```

Figure 7: Execution log

```
theusefulhacker@debian:~/project_root$ ./server missing.dat
Cannot read input file: missing.dat
theusefulhacker@debian:~/project_root$ ./server original.dat
bind: Address already in use
theusefulhacker@debian:~/project_root$
```

Figure 8: Error handling tests

Part 2

```
~/Desktop/os latest  cat result_max.txt
MAX=2147483259
~/Desktop/os latest  cat result_min.txt
MIN=-2147483520
~/Desktop/os latest  cat execution_log.txt
[PART1] CHUNKS=4 | PROCS=4 | SYNC_USED=mutex,sem,condvar
[PART2] THREADS=8 | DATA_PARALLEL=min,max | TASK_PARALLEL=sort
[PART2] TIME_MS=4264 | SORT_ALGO=parallel_merge_sort
[STATUS] SUCCESS
```

Figure 9: Min/Max Computation Results

Validation: With random int32_t data, min/max should approach INT32_MIN/INT32_MAX

```
~/Desktop/os latest  for threads in 1 2 4 8; do
for> echo -n "T=$threads: "
for> start=$(date +%s%N)
for> ./operations -t $threads -f reassembled.dat 2>&1 >/dev/null
for> end=$(date +%s%N)
for> elapsed=$(( (end - start) / 1000000 ))
for> echo "${elapsed}ms"
for> done
T=1: 15323ms
T=2: 12817ms
T=4: 10617ms
T=8: 8706ms
```

Figure 10: Thread timing comparison (T=1, 2, 4, 8)

Challenges & Solutions

Challenge	Risk	Solution
Partial socket transfer	Both send() and recv() may process only part of a buffer	I implemented send_all() and recv_all() to help get loop until the full header or payload is transferred
Out of order chunk arrival	Parallel child processes can finish in any order	I used the sequence number from the chunk header and wrote to (sequence – 1) x chunk_size
Process-shared synchronization	Default pthread_mutex_t and pthread_cond_t only work within a single process. Using them across fork()ed children causes deadlocks.	I initialized the primitives with pthread_mutexattr_setpshared() using PTHREAD_PROCESS_SHARED so they work across the mmap'd shared memory.
Memory allocation for >1GB files	Using standard malloc() for a 1GB+ contiguous buffer can fail due to heap fragmentation or OS limits.	I used mmap() with MAP_SHARED | MAP_ANONYMOUS to allocate the reassembly buffer directly from the OS virtual memory, which is safer for massive files.
Merge sort level coordination	In bottom-up merge sort, level N+1 cannot start until level N finishes. Spawning too many merge threads also causes context-switch overhead.	I used a pthread_cond_t as a barrier to block until all merges at level N finish. I also used a sem_t to throttle active merge tasks to exactly T threads.

References

- Week 3 lecture notes, page 16, 18, 20-22: process creation, fork(), exec(), wait(), and process termination. Used for server child processes, client children, and launching operations.
- Week 3 lecture notes, page 28-33, 38-40: interprocess communication, producer-consumer model, shared memory, and message passing. Used for the server-producer/client-consumer design and chunk communication model.
- Week 3 lecture notes, page 36-37: shared-memory and mmap-style examples. Used for the client shared reassembly buffer design.
- Week 4 lecture notes, page 4-12: multithreaded processes, concurrency vs parallelism, data parallelism, and task parallelism. Used for the operations min/max and sort design.
- Week 4 lecture notes, page 21-22, 26: POSIX pthreads and fork-join parallelism. Used for pthread_create(), pthread_join(), and worker coordination.
- Week 5 lecture notes, page 26-35: pthread scheduling, multicore processors, and multiple-processor scheduling. Used as background for running multiple analytics threads.
- Week 6 lecture notes, page 4-9: race conditions and critical-section problem. Used to justify protecting shared counters, buffers, and min/max updates.
- Week 6 lecture notes, page 29-41: mutex locks, POSIX mutex examples, semaphores, POSIX semaphores, monitors, and condition variables. Used for mutex, sem_t, and pthread_cond_t synchronization.
- Week 7 lecture notes, page 3, 11-18: deadlock definition and deadlock prevention. Used as background for avoiding unsafe lock ordering and indefinite waits.
- Week 9/10 lecture notes, page 2, 22: memory-mapped files and copy-on-write. Used as background for mmap and forked-process memory behavior.
- Week 11 lecture notes, page 6-8, 15, 32-35: file operations, open files, file locking, direct access, file sharing, remote file systems, and consistency semantics. Used for open/read/write/seek and file-transfer correctness.
- Week 12 lecture notes, page 3, 10, 22, 30-31, 43-46, 52-55: file-system structure, in-memory file structures, performance, page cache, remote file systems, client-server model, NFS protocol, and remote operations. Used for large-file I/O and client-server file movement.
- Week 13 lecture notes, page 24, 40, 58-60, 62, 70-71: error detection/checksums, network sockets, memory-mapped file access, blocking/nonblocking I/O, vectored I/O, and kernel I/O data structures. Used for MD5 validation and socket/file-I/O explanation.
- Week 14 lecture notes, page 9-12, 57-62: UNIX system calls, file manipulation, process control, pipes/sockets, socket types, socket system calls, and network support. Used for the server/client socket API explanation.